Unidad 1. MANEJO DE FICHEROS

ACCESO A DATOS

IES Francisco de Goya

Curso 2017/2018

1. Clases asociadas a las operaciones de gestión de ficheros

El paquete java.io contiene las clases para manejar la entrada/salida en Java, por tanto necesitaremos importar dicho paquete cuando trabajemos con ficheros.

Para crear un objeto File se pueden utilizar cualquiera de estos tres constructores:

- File(String directorioyfichero): en Linux: new File("/directorio/ficheroi.txt"); en Windows new Filw("C:\\directorio\fichero.txt")
- File(String directorio, String nombreFichero)
- File(File directorio, String fichero)

En Linux se utiliza como prefijo de una ruta absoluta "/"; en windows el prefijo de un nombre de ruta consiste en la letra de la unidad seguida de ":" y posiblemente, seguido de "\\" si la ruta es absoluta.

1. Clases asociadas a las operaciones de gestión de ficheros

MÉTODO	FUNCIÓN
boolean canRead()	Devuelve true si se puede leer el fichero
boolean canWrite()	Devuelve true si se puede escribir en el fichero
boolean createNewFile()	Crea el fichero asociado al objeto File. Devuelve true si se ha podido crear. Para poder crearlo el fichero no debe existir. Lanza una excepción del tipo IOException.
boolean delete()	Elimina el fichero o directorio. Si es un directorio debe estar vacío. Devuelve true si se ha podido eliminar.
boolean exists()	Devuelve true si el fichero o directorio existe
String getName()	Devuelve el nombre del fichero o directorio
String getAbsolutePath()	Devuelve la ruta absoluta asociada al objeto File.
String getCanonicalPath()	Devuelve la ruta única absoluta asociada al objeto File. Puede haber varias rutas absolutas asociadas a un File pero solo una única ruta canónica. Lanza una excepción del tipo IOException.
String getPath()	Devuelve la ruta con la que se creó el objeto File. Puede ser relativa o no.

1. Clases asociadas a las operaciones de gestión de ficheros

MÉTODO	FUNCIÓN
String getParent()	Devuelve un String conteniendo el directorio padre del File. Devuelve null si no tiene directorio padre.
File getParentFile()	Devuelve un objeto File conteniendo el directorio padre del File. Devuelve null si no tiene directorio padre.
boolean isAbsolute()	Devuelve true si es una ruta absoluta
boolean isDirectory()	Devuelve true si es un directorio válido
boolean isFile()	Devuelve true si es un fichero válido
long lastModified()	Devuelve un valor en milisegundos que representa la última vez que se ha modificado (medido desde las 00:00:00 GMT, del 1 de Enero de 1970). Devuelve 0 si el fichero no existe o ha ocurrido un error.
long length()	Devuelve el tamaño en bytes del fichero. Devuelve 0 si no existe. Devuelve un valor indeterminado si es un directorio.
String[] list()	Devuelve un array de String con el nombre de los archivos y directorios que contiene el directorio indicado en el objeto File. Si no es un directorio devuelve null. Si el directorio está vacío devuelve un array vacío.
String[] list(FilenameFilter filtro)	Similar al anterior. Devuelve un array de String con el nombre de los archivos y directorios que contiene el directorio indicado en el objeto File que cumplen con el filtro indicado.
boolean mkdir()	Crea el directorio. Devuelve true si se ha podido crear.
boolean mkdirs()	Crea el directorio incluyendo los directorios no existentes especificados en la ruta padre del directorio a crear. Devuelve true si se ha creado el directorio y los directorios no existentes de la ruta padre.
boolean renameTo(File dest)	Cambia el nombre del fichero por el indicado en el parámetro dest. Devuelve true si se ha realizado el cambio.

2. Formas de acceso a un fichero

Hay dos formas de acceso a la información almacenada en un fichero: acceso secuencial y acceso directo o aleatorio:

- Acceso secuencial: los datos o registro se leen y se escriben en orden, del mismo modo que se hace en una antigua cinta de video. Si se quiere acceder a un dato o un registro que está hacia la mitad del fichero es necesario leer antes todos los anteriores.
- Acceso directo o aleatorio: permite acceder directamente a un dato o registro sin necesidad de leer los anteriores y se puede acceder a la información en cualquier orden. Los datos están almacenados en registros de tamaño conocido, nos podemos mover de un registro a otro de forma aleatoria para leerlos o modificarlos.

En Java el acceso secuencial más común en ficheros puede ser binario o a caracteres. Para el acceso binario: se usan las clases **FileInputStream** y **FileOutputStream**; para el acceso a caracteres se usan las clases **FileReader** y **FileWriter**. En el accedo aleatorio se utiliza la clase **RandomAccessFile**.

Ficheros de texto

- **FileReader**: para leer caracteres

  Genera la excepción FileNotFoundException (porque el nombre del fichero no exista o no sea válido)

- **FileWriter**: para escribir caracteres

  Genera la excepción IOException (porque el disco está lleno o protegido contra escritura)

Ficheros binarios: trabajan con flujo de bytes y el fichero.

- **FileInputStream**: para entrada
- **FileOutputStream**: para salida

2. Formas de acceso a un fichero

Métodos proporcionados por la clase **FileReader**: devuelven el número de caracteres leídos o -1 si se ha llegado al final del fichero.

MÉTODO	FUNCIÓN
int read()	Lee un carácter y lo devuelve
int read(char[] buf)	Lee hasta *buf.length* caracteres de datos de una matriz de caracteres (*buf*). Los caracteres leídos del fichero se van almacenando en *buf*
int read(char[] buf, int desplazamiento, int n)	Lee hasta n caracteres de datos de la matriz *buf* comenzando por *buf[desplazamiento]* y devuelve el número leído de caracteres

Métodos proporcionados por la clase **FileWriter**:

MÉTODO	FUNCIÓN
void write(int c)	Escribe un carácter
void write(char[] buf)	Escribe un array de caracteres
void write(char[] buf, int desplazamiento, int n)	Escribe n caracteres de datos en la matriz *buf* comenzando por *buf[desplazamiento]*
void write(String str)	Escribe una cadena de caracteres
void append(char c)	Añade un carácter a un fichero

2. Formas de acceso a un fichero

Ficheros binarios

Métodos proporcionados por la clase **FileInputStream** para lectura: devuelven el número de caracteres leídos o -1 si se ha llegado al final del fichero.

MÉTODO	FUNCIÓN
int read()	Lee un byte y lo devuelve
int read(byte b)	Lee hasta *b.length* bytes de una matriz de bytes
int read(byte[] b, int desplazamiento, int n)	Lee hasta n bytes de la matriz *b* comenzando por *b[desplazamiento]* y devuelve el número leído de bytes

Métodos proporcionados por la clase **FileOutputStream** para escritura:

MÉTODO	FUNCIÓN
void write(int b)	Escribe un byte
void write(byte[] b)	Escribe b.length bytes
void write(byte[] b, int desplazamiento, int n)	Escribe n n bytes a partir de la matriz de bytes de entrada comenzando por *b[desplazamiento]*

Para añadir bytes al final del fichero usaremos FileOutputStream, pondremos como valor del segundo parámetro **true**:

FileOutputStream fileout = new FileOutputStream (fichero, true);

2. Formas de acceso a un fichero

Ficheros binarios

Para leer y escribir datos de tipos primitivos: int, float, long, etc., se utilizarán las clases DataInputStream y DataOutputStream, mediante los métodos readXXX y writeXXX, que son variaciones de los métodos read() y write():

MÉTODOS PARA LECTURA	MÉTODOS PARA ESCRITURA
boolean readBoolean();	void writeBoolean(boolean v);
byte readByte();	void writeByte(int v);
int readUnsignedByte();	void writeBytes(String s);
int readUnsignedShort();	void writeShort(int v);
int readShort();	void writeChars(String s);
char readChar();	void writeChar(int v);
int readInt();	void writeInt(int v);
long readLong();	void writeLong(long v);
float readFloat();	void writeFloat(float v);
double readDouble();	void writeDouble(double v);
String readUTF();	void writeUTF(String str);

2. Formas de acceso a un fichero

Ficheros binarios

Para abrir un objeto DataInputStream, se utilizan los mismos métodos que para FileInputStream:

```
File fichero=new File("FichData.dat");
File InputStream filein= new FileInputStream(fichero);
DataInputStream dataIS= new DataInputStream(filein);
```

O bien;

```
File fichero= new File("FichData.dat");
DataInputStream dataIS= new DataInputStream(new FileInputStream(fichero));
```

Lo mismo para un objeto DataOutputStream (mismos métodos que para FileOutputStream):

```
File fichero=new File("FichData.dat");
File OutputStream fileout= new FileOutputStream(fichero);
DataOutputStream dataOS= new DataOutputStream(filein);
```

O bien;

```
File fichero= new File("FichData.dat");
DataOutputStream dataOS= new DataOutputStream(new FileOutputStream(fichero));
```

2. Formas de acceso a un fichero

Objetos en ficheros binarios

Hemos visto cómo guardar datos primitivos, pero si queremos guardar en un fichero por ejemplo, un objeto de tipo empleado(nombre, dirección, salario, etc.), tendríamos que guardar cada atributo que forma parte del objeto por separado, y eso es engorroso. Java nos permite guardar objetos en ficheros binarios, implementando la interfaz **Serializable**, que dispone de métodos que nos permiten guardar y leer objetos en ficheros binarios.

- **Object readObjetc()**: se utiliza para leer un objeto del ObjectInputStream. Puede lanzar las excepciones IOException y **ClassNotFoundException**.
- **void writeObject(Object obj)**: se utiliza para escribir el objeto especificado en el ObjectOutputStream. Puede lanzar la excepción **IOException**.

La serialización de objetos de Java permite tomar cualquier objeto que implemente la interfaz Serializable y convertirlo en una secuencia de bits que puede ser posteriormente restaurada para regenerar el objeto original.

Para leer y escribir objetos serializables a un stream se utilizan las clases Java **ObjectInputStream** y **ObjectOutputStream**, respectivamente.

2. Formas de acceso a un fichero

Ficheros de acceso aleatorio

En ficheros secuenciales, la lectura se hacía desde el primer byte, carácter, objeto, leyendo uno a uno hasta llegar al fin del fichero. Lo mismo en escritura; Java dispone de la clase RandomAccessFile que tiene métodos para acceder al contenido de un fichero binario de forma aleatoria y para posicionarnos en una posición concreta del mismo.

Existen dos constructores para crear el fichero de acceso aleatorio, los cuales pueden lanzar la excepción **FileNotFoundException**:

- **RandomAccessFile (String nombreFichero, String modoAcceso)**: escribiendo el nombre del fichero, incluído el path.
- **RandomAccessFile (File objectoFile, String modoAcceso)**: con un objeto File asociado a un fichero.

El argumento **modoAcceso** puede tener dos valores:

- **r**: abre el fichero en modo de solo lectura. El fichero debe existir. Una operación de escritura en este fichero lanzará la excepción **IOException**.
- **rw**: abre el fichero en modo lectura y escritura. Si el fichero no existe, se crea.

Una vez abierto el fichero pueden usarse los métodos readXXX y writeXXX de las clases DataInputStream y DataOutputStream.

La clase **RandomAccessFile** maneja un puntero que indica la posición actual en el fichero; cuando el fichero se crea, el puntero al fichero se coloca en 0, apuntando al principio del mismo. Las sucesivas llamadas a los métodos read() y write() ajustan el puntero según la cantidad de bytes leídos o escritos.

2. Formas de acceso a un fichero

Ficheros de acceso aleatorio

Los métodos más importantes son:

MÉTODO	FUNCIÓN
long getFilePointer()	Devuelve la posición actual del puntero del fichero
void seek (long posición)	Coloca el puntero del fichero en una posición determinada desde el comienzo del mismo
long length()	Devuelve el tamaño del fichero en bytes. La posición length() marca el final del fichero
int skipBytes (int desplazamiento)	Desplaza el puntero desde la posición actual el número de bytes indicados en desplazamiento